

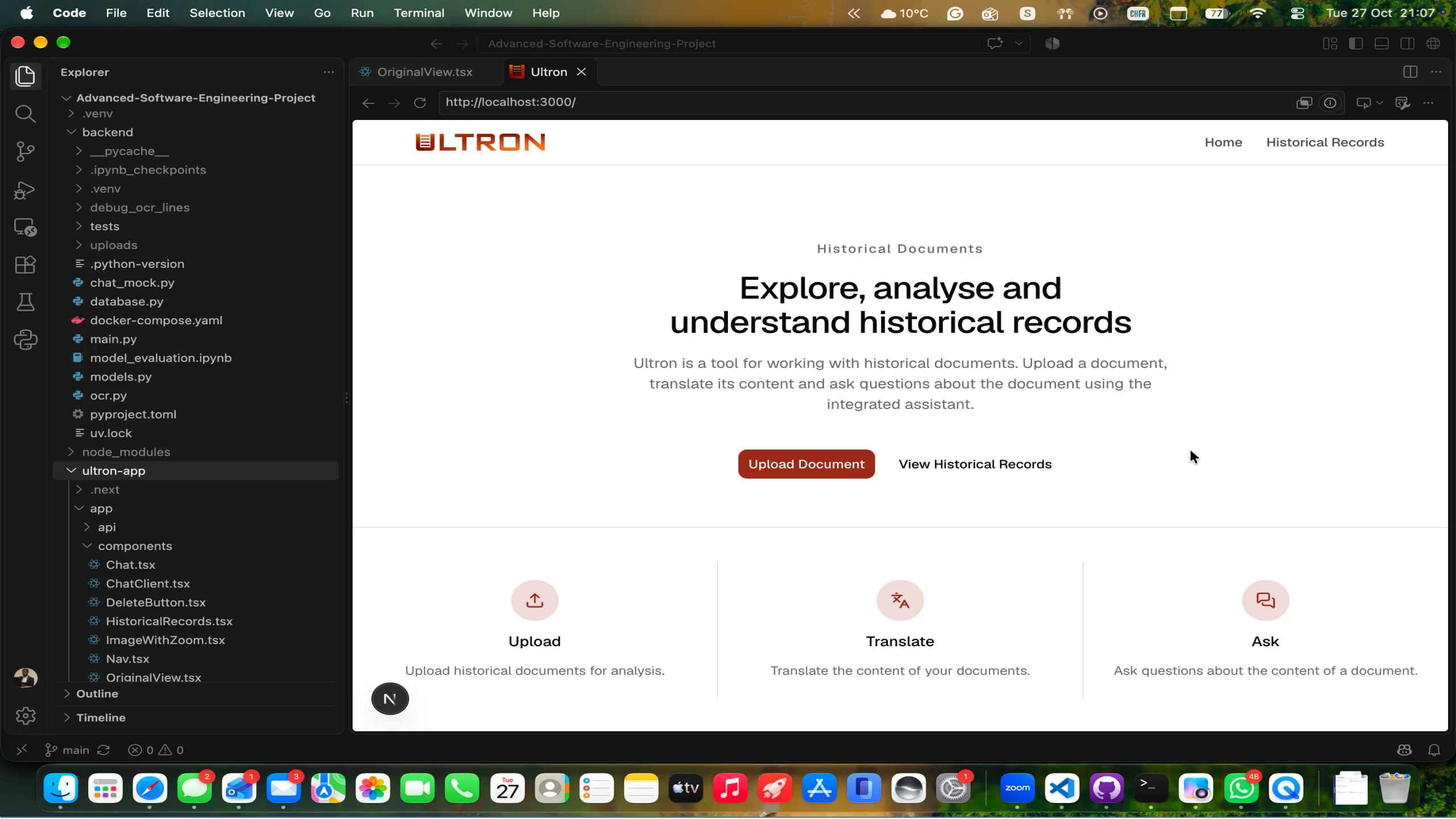
Advanced Software Engineering Project

Elisa, Hans, Jules, Richmond, Valery, Viola

Introduction

From image to understanding

1. **Upload** a photo of a historical document
2. **Transcribe** the handwriting automatically
3. **Translate** the text into English
4. **Ask** questions about the document in a chat


Code File Edit Selection View Go Run Terminal Window Help
10°C
Tue 27 Oct 21:07
Advanced-Software-Engineering-Project
Explorer
Advanced-Software-Engineering-Project
.venv
backend
__pycache__
.ipynb_checkpoints
.venv
debug_ocr_lines
tests
uploads
.python-version
chat_mock.py
database.py
docker-compose.yaml
main.py
model_evaluation.ipynb
models.py
ocr.py
pyproject.toml
uv.lock
node_modules
ultron-app
.next
app
api
components
Chat.tsx
ChatClient.tsx
DeleteButton.tsx
HistoricalRecords.tsx
ImageWithZoom.tsx
Nav.tsx
OriginalView.tsx
Outline
Timeline
OriginalView.tsx
Ultron
http://localhost:3000/
ULTRON
Home
Historical Records
Historical Documents
Explore, analyse and understand historical records
Ultron is a tool for working with historical documents. Upload a document, translate its content and ask questions about the document using the integrated assistant.
Upload Document
View Historical Records
Upload
Upload historical documents for analysis.
Translate
Translate the content of your documents.
Ask
Ask questions about the content of a document.
main
0 0

Technology Stack

Ultron · historical document transcription

FRONTEND

Next.js · React

TypeScript

Tailwind CSS · shadcn/ui · Lucide

BACKEND + OCR

Python · FastAPI · Uvicorn

SQLAlchemy · PostgreSQL

TrOCR · PyTorch · Transformers

DATA + TOOLS

Docker Compose

npm · uv · Jupyter

PostgresSQL

IMAGE-TO-TRANSCRIPTION FLOW

Image upload → **Next.js UI** → **FastAPI + TrOCR** → **Image file + PostgreSQL record**

MODEL EVALUATION

TrOCR vs Qwen2-VL on CATMuS/medieval .
Character Error Rate (CER) and Word Error Rate (WER) measured with JiWER

Languages: ['Castilian', 'Catalan', 'French', 'Italian', 'Latin']

```
TrOCR
Successful predictions: 300/300
CER: 18.83%
WER: 54.42%

Qwen2-VL
Successful predictions: 300/300
CER: 25.47%
WER: 69.90%
```

Continuous Integration

Automated quality checks from change to merge

01 **A change starts the workflow**

Runs on pushes, pull requests, and pull-request review events.

02 **Pull requests get additional safeguards**

Checks the title and description sections, then requires independent approval of the latest commit.

03 **Backend and frontend checks run**

Each job installs locked dependencies and runs its own quality checks, tests, and build validation.

04 **The team acts on the result**

A failed check points to an issue to fix; a passing run gives repeatable evidence for review.

Backend CI

Python 3.14 / FastAPI

01 **Reproducible environment**

uv sync --frozen installs the versions recorded in the backend lockfile.

02 **Style and static checks**

Ruff checks formatting and lint; mypy checks the backend modules for type errors.

03 **API integration tests**

Six pytest cases exercise FastAPI routes with an isolated in-memory SQLite database.

04 **Python syntax check**

Compileall checks that the main application modules compile successfully.

Frontend CI

Next.js / React / TypeScript

01 **Reproducible environment**

npm ci installs the frontend dependencies from package-lock.json.

02 **Format, type, and lint checks**

Prettier checks formatting; Next typegen and TypeScript check types; ESLint checks code rules.

03 **Frontend unit tests**

Three Vitest tests cover chat-service behavior; the backend API call is mocked.

04 **Production build**

Next.js builds the app to catch production compilation errors.

What does a green CI run mean?

Fast feedback before merge not a guarantee that the application is bug-free

01 **Defined checks passed for this change**

Formatting, lint, type checks, automated tests, syntax validation, and frontend build completed successfully.

02 **The tests cover specific behavior**

Backend route behavior is tested; frontend chat logic is tested in isolation.

03 **Some testing is not included yet**

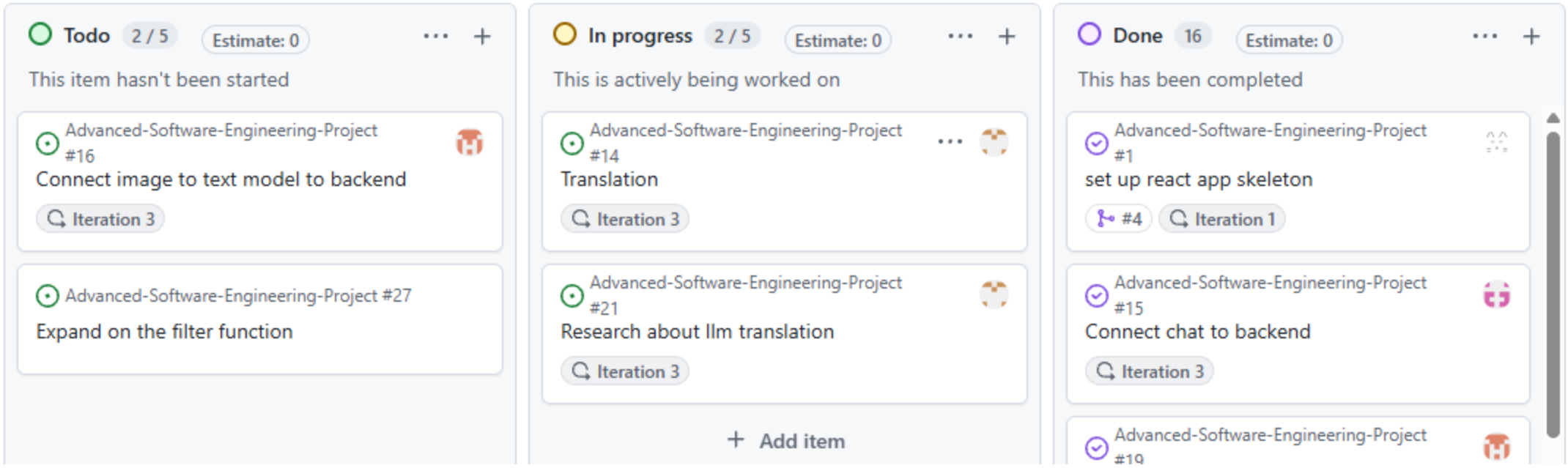
There are no browser end-to-end tests, and backend tests do not use a live PostgreSQL database.

04 **The purpose**

Catch regressions early and make code review and delivery more reliable.

User Stories

- 2 Meetings a week
- GitHub project used for user stories



Future Work

- Optimizing the image to text algorithm.
- Show the transcription lines.
- Optimizing the English translation.
- Chat LLM (gives summary of the document is about and give defined question options). Source will be document only.
- Visible feedback for loading data and transcribing.
- User logins and data storage?

Q & A